Algorithm for File Updates in Python

Simulated cybersecurity training project | Google Cybersecurity Professional Certificate

Project description

In this simulated healthcare security scenario, access to a restricted subnetwork is controlled using an IP address allow list. A separate remove list identifies addresses that should no longer be permitted. I developed a Python algorithm to read the allow-list file, compare its entries against the remove list, remove matching addresses, and write the revised list back to the file. This exercise demonstrates file handling, list manipulation, iteration, and conditional logic in support of access-control administration.

Open the file that contains the allow list

The filename allow_list.txt is assigned to import_file. The with open(import_file, "r") as file statement opens the existing file in read mode and automatically closes it when the block ends.

Read the file contents

The file.read() method retrieves the contents as a single string, which is assigned to ip_addresses.

Convert the string into a list

The ip_addresses.split() method separates whitespace-delimited addresses into individual list elements so each address can be checked and removed.

Iterate through the remove list

The for element in remove_list: loop examines each IP address scheduled for removal.

Remove IP addresses that are on the remove list

The condition if element in ip_addresses: checks for an exact matching list entry. When present, ip_addresses.remove(element) removes it. This approach is sufficient for the course scenario because the allow list is specified to contain no duplicate IP addresses; remove() removes only the first occurrence.

Update the file with the revised list of IP addresses

The expression "\n".join(ip_addresses) converts the remaining addresses to newline-separated text. Opening the original filename with "w" and calling file.write(ip_addresses) replaces its previous contents with the revised list. Write mode truncates the existing file, so the algorithm should be tested on a copy first.

Complete Python algorithm

```
"""Simulated cybersecurity training: update an IP allow-list file."""

import_file = "allow_list.txt"

with open(import_file, "r", encoding="utf-8") as file:
    ip_addresses = file.read()

ip_addresses = ip_addresses.split()

# Illustrative test values; replace with the authorized removal list.
remove_list = ["192.168.1.20", "192.168.1.40"]
```

```
for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)

ip_addresses = "\n".join(ip_addresses)

with open(import_file, "w", encoding="utf-8") as file:
    file.write(ip_addresses)
```

Local validation (illustrative data)

Test input: 192.168.1.10, 192.168.1.20, 192.168.1.30, 192.168.1.40 (one per line). Illustrative remove list: 192.168.1.20 and 192.168.1.40. Expected and observed output: 192.168.1.10 and 192.168.1.30 (one per line). This local test is not evidence that the original Google lab was executed.

Summary

The algorithm reads an IP address allow-list file and converts its contents into a Python list. It iterates through a separate remove list and checks each address for membership in the allow list. Matching addresses are removed using the list remove() method. The remaining addresses are joined into a newline-separated string and written back to the original file. A local test with illustrative data verifies the expected file update. Updating the file alone does not prove that network controls have applied the changes.